

## **EXPERIMENT – 6**

**AIM:** To create a stored procedure `transfer_employee(emp_id, new_dept_id)` with validation and error handling, and to implement database triggers for salary validation and audit logging. The procedure and triggers are tested using normal and edge-case inputs.

### **SOURCE CODE:**

#### **USING EXPERIMENT – 3 DATABASE**

### **SQL QUERIES:**

**STORED PROCEDURE:** `transfer_employee`

The procedure will:

1. Check whether the employee exists.
2. Check whether the new department exists.
3. Check whether the employee is already in that department.
4. Transfer the employee.
5. Handle errors using `SQLEXCEPTION`.
6. Roll back the transaction if an error occurs.

```
DELIMITER //
```

```
CREATE PROCEDURE transfer_employee(  
    IN p_emp_id INT,  
    IN p_new_dept_id INT  
)  
BEGIN  
    DECLARE v_employee_count INT DEFAULT 0;  
    DECLARE v_department_count INT DEFAULT 0;  
    DECLARE v_current_dept INT DEFAULT 0;  
  
    -- Error handler  
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
    BEGIN  
        ROLLBACK;  
        SELECT 'Error occurred. Transaction rolled back.' AS  
message;  
        END;  
  
    START TRANSACTION;
```

```

-- Check whether employee exists
SELECT COUNT(*)
INTO v_employee_count
FROM Employee
WHERE emp_id = p_emp_id;

IF v_employee_count = 0 THEN
    ROLLBACK;
    SELECT 'Error: Employee does not exist.' AS message;

ELSE

    -- Check whether department exists
    SELECT COUNT(*)
    INTO v_department_count
    FROM Department
    WHERE dept_id = p_new_dept_id;

    IF v_department_count = 0 THEN
        ROLLBACK;
        SELECT 'Error: Department does not exist.' AS
message;

    ELSE

        -- Get current department
        SELECT dept_id
        INTO v_current_dept
        FROM Employee
        WHERE emp_id = p_emp_id;

        -- Check if already in requested department
        IF v_current_dept = p_new_dept_id THEN
            ROLLBACK;
            SELECT 'Error: Employee is already in this
department.'
                AS message;

        ELSE

            -- Transfer employee
            UPDATE Employee
            SET dept_id = p_new_dept_id
            WHERE emp_id = p_emp_id;

            COMMIT;

            SELECT
                'Employee transferred successfully.'
                AS message;

```

```

        END IF;

    END IF;

END IF;

END //

DELIMITER ;

```

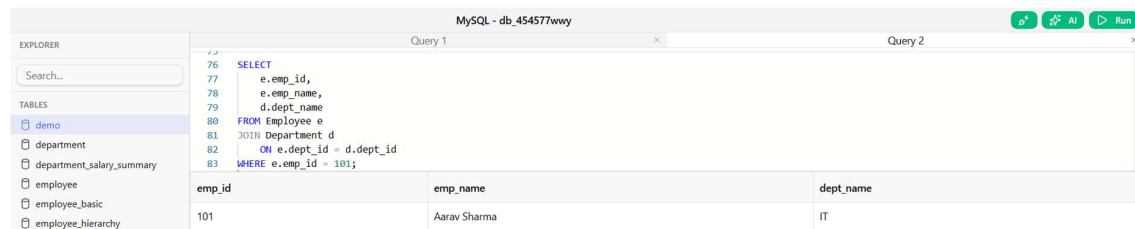
## TESTING THE STORED PROCEDURE

```

SELECT
    e.emp_id,
    e.emp_name,
    d.dept_name
FROM Employee e
JOIN Department d
    ON e.dept_id = d.dept_id
WHERE e.emp_id = 101;

```

## OUTPUT:

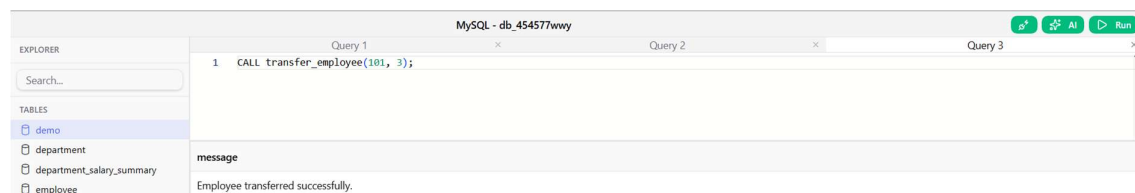


The screenshot shows the MySQL IDE interface. The 'EXPLORER' panel on the left lists tables: demo, department, department\_salary\_summary, employee, employee\_basic, and employee\_hierarchy. The 'demo' table is selected. The main editor shows a SQL query (Query 1) with line numbers 76 to 83. The query is a SELECT statement joining Employee and Department tables, filtering for emp\_id = 101. Below the query editor, the results are displayed in a table with three columns: emp\_id, emp\_name, and dept\_name. The result shows emp\_id 101, emp\_name 'Aarav Sharma', and dept\_name 'IT'.

| emp_id | emp_name     | dept_name |
|--------|--------------|-----------|
| 101    | Aarav Sharma | IT        |

```
CALL transfer_employee(101, 3);
```

## OUTPUT:



The screenshot shows the MySQL IDE interface. The 'EXPLORER' panel on the left lists tables: demo, department, department\_salary\_summary, and employee. The 'demo' table is selected. The main editor shows a SQL query (Query 1) with line number 1. The query is a CALL statement: CALL transfer\_employee(101, 3);. Below the query editor, the results are displayed in a table with one column: message. The result shows the message 'Employee transferred successfully.'

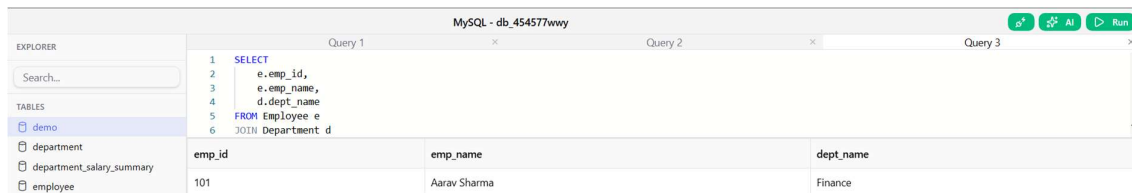
| message                            |
|------------------------------------|
| Employee transferred successfully. |

```

SELECT
    e.emp_id,
    e.emp_name,
    d.dept_name
FROM Employee e
JOIN Department d
    ON e.dept_id = d.dept_id
WHERE e.emp_id = 101;

```

## OUTPUT:



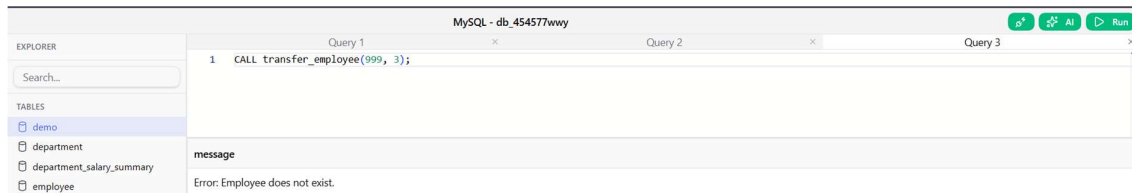
The screenshot shows the MySQL IDE interface. The 'EXPLORER' panel on the left lists tables: demo, department, department\_salary\_summary, and employee. The 'Query 1' tab is active, displaying the SQL query. The results pane shows a single row with columns emp\_id, emp\_name, and dept\_name.

| emp_id | emp_name     | dept_name |
|--------|--------------|-----------|
| 101    | Aarav Sharma | Finance   |

## EDGE CASE – EMPLOYEE DOES NOT EXIST

```
CALL transfer_employee(999, 3);
```

## OUTPUT:



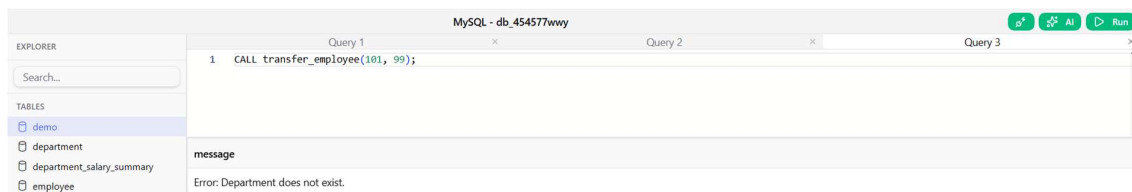
The screenshot shows the MySQL IDE interface. The 'Query 1' tab is active, displaying the SQL query. The results pane shows a message indicating an error.

| message                         |
|---------------------------------|
| Error: Employee does not exist. |

## EDGE CASE – DEPARTMENT DOES NOT EXIST

```
CALL transfer_employee(101, 99);
```

## OUTPUT:



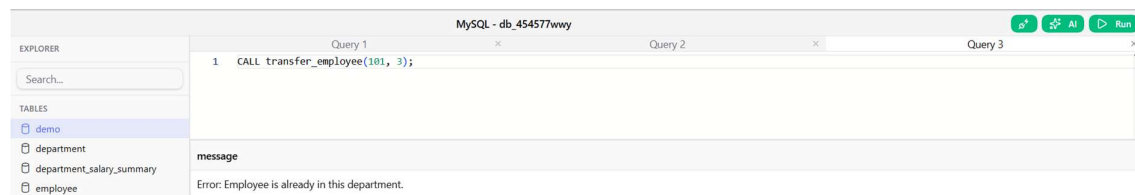
The screenshot shows the MySQL IDE interface. The 'Query 1' tab is active, displaying the SQL query. The results pane shows a message indicating an error.

| message                           |
|-----------------------------------|
| Error: Department does not exist. |

## EDGE CASE – SAME DEPARTMENT

```
CALL transfer_employee(101, 3);
```

### OUTPUT:



## SALARY VALIDATION TRIGGER

Let, salary must be  $\geq 15000$

```
DELIMITER //
```

```
CREATE TRIGGER validate_employee_salary
BEFORE INSERT ON Employee
FOR EACH ROW
BEGIN
    IF NEW.salary < 15000 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary must be at least 15000';
    END IF;
```

```
END //
```

```
DELIMITER ;
```

## TEST SALARY VALIDATION

```
INSERT INTO Employee
(emp_id, emp_name, gender, salary, hire_date, dept_id)
VALUES
(131, 'Test Employee', 'M', 10000, '2026-09-25', 1);
```

### OUTPUT:

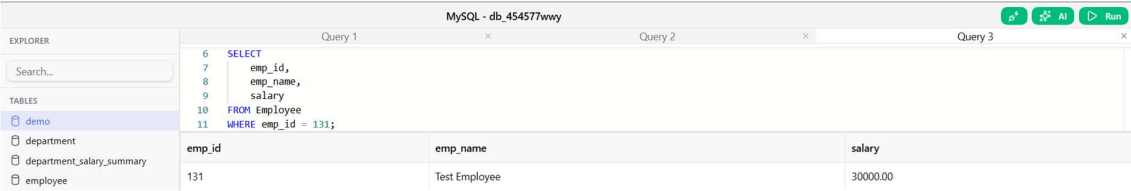


## TEST VALID SALARY

```
INSERT INTO Employee
(emp_id, emp_name, gender, salary, hire_date, dept_id)
VALUES
(131, 'Test Employee', 'M', 30000, '2026-09-25', 1);

SELECT
    emp_id,
    emp_name,
    salary
FROM Employee
WHERE emp_id = 131;
```

## OUTPUT:



The screenshot shows a MySQL IDE window titled 'MySQL - db\_454577wwy'. On the left is an 'EXPLORER' pane with a search bar and a 'TABLES' list containing 'demo', 'department', 'department\_salary\_summary', and 'employee'. The 'demo' table is selected. The main area shows 'Query 1' with the following SQL code:

```
6 SELECT
7   emp_id,
8   emp_name,
9   salary
10 FROM Employee
11 WHERE emp_id = 131;
```

Below the query, the results are displayed in a table with three columns: 'emp\_id', 'emp\_name', and 'salary'. The results show one row with the values 131, Test Employee, and 30000.00.

| emp_id | emp_name      | salary   |
|--------|---------------|----------|
| 131    | Test Employee | 30000.00 |

## SALARY VALIDATION FOR UPDATE

```
DELIMITER //
```

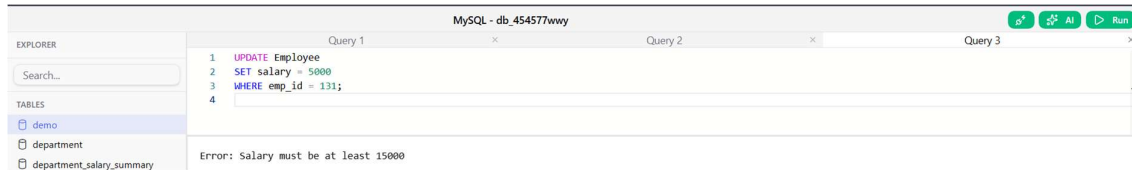
```
CREATE TRIGGER validate_employee_salary_update
BEFORE UPDATE ON Employee
FOR EACH ROW
BEGIN
    IF NEW.salary < 15000 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary must be at least 15000';
    END IF;
END //
```

```
DELIMITER ;
```

## TEST INVALID SALARY UPDATE

```
UPDATE Employee
SET salary = 5000
WHERE emp_id = 131;
```

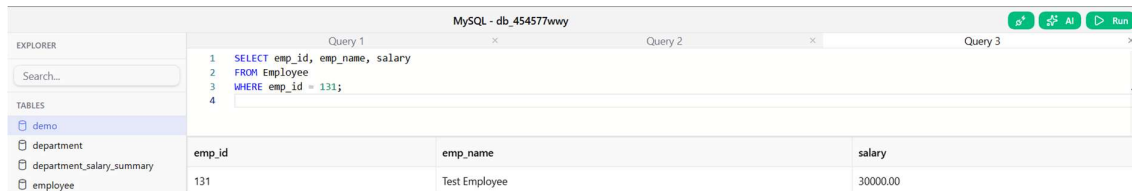
### OUTPUT:



The screenshot shows the MySQL Workbench interface. The 'EXPLORER' pane on the left lists tables: demo, department, and department\_salary\_summary. The 'Query 1' pane contains the SQL statement: `UPDATE Employee SET salary = 5000 WHERE emp_id = 131;`. The 'Query 3' pane displays the error message: 'Error: Salary must be at least 15000'.

```
SELECT emp_id, emp_name, salary
FROM Employee
WHERE emp_id = 131;
```

### OUTPUT:



The screenshot shows the MySQL Workbench interface. The 'EXPLORER' pane on the left lists tables: demo, department, department\_salary\_summary, and employee. The 'Query 1' pane contains the SQL statement: `SELECT emp_id, emp_name, salary FROM Employee WHERE emp_id = 131;`. The 'Query 3' pane displays the output of the query as a table with three columns: emp\_id, emp\_name, and salary. The output shows a single row with emp\_id 131, emp\_name 'Test Employee', and salary 30000.00.

| emp_id | emp_name      | salary   |
|--------|---------------|----------|
| 131    | Test Employee | 30000.00 |

## CREATE AUDIT TABLE

```
CREATE TABLE Salary_Audit (
    audit_id INT AUTO_INCREMENT PRIMARY KEY,
    emp_id INT,
    old_salary DECIMAL(10,2),
    new_salary DECIMAL(10,2),
    changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    action VARCHAR(30)
);
```

## AUDIT LOGGING TRIGGER

```
DELIMITER //
```

```
CREATE TRIGGER salary_audit_trigger
AFTER UPDATE ON Employee
FOR EACH ROW
BEGIN
```

```

IF OLD.salary <> NEW.salary THEN

    INSERT INTO Salary_Audit
    (
        emp_id,
        old_salary,
        new_salary,
        action
    )
VALUES
    (
        NEW.emp_id,
        OLD.salary,
        NEW.salary,
        'SALARY UPDATED'
    );

END IF;

END //

DELIMITER ;

```

### TEST AUDIT TRIGGER

```

UPDATE Employee
SET salary = 35000
WHERE emp_id = 131;

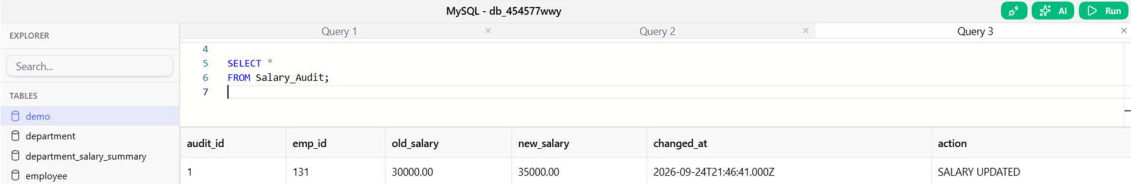
```

```

SELECT *
FROM Salary_Audit;

```

### OUTPUT:



The screenshot shows a MySQL database interface with a table named 'Salary\_Audit'. The table has columns: audit\_id, emp\_id, old\_salary, new\_salary, changed\_at, and action. The data shows a salary update for employee 131 from 30000.00 to 35000.00 on 2026-09-24T21:46:41.000Z.

| audit_id | emp_id | old_salary | new_salary | changed_at               | action         |
|----------|--------|------------|------------|--------------------------|----------------|
| 1        | 131    | 30000.00   | 35000.00   | 2026-09-24T21:46:41.000Z | SALARY UPDATED |

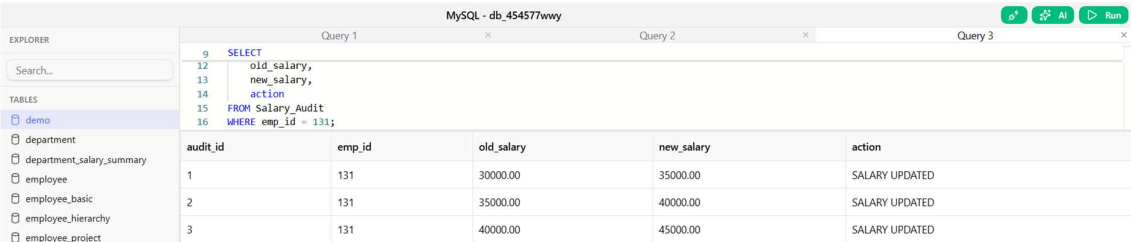
## TEST MULTIPLE SALARY CHANGES

```
UPDATE Employee
SET salary = 40000
WHERE emp_id = 131;
```

```
UPDATE Employee
SET salary = 45000
WHERE emp_id = 131;
```

```
SELECT
    audit_id,
    emp_id,
    old_salary,
    new_salary,
    action
FROM Salary_Audit
WHERE emp_id = 131;
```

## OUTPUT:



| audit_id | emp_id | old_salary | new_salary | action         |
|----------|--------|------------|------------|----------------|
| 1        | 131    | 30000.00   | 35000.00   | SALARY UPDATED |
| 2        | 131    | 35000.00   | 40000.00   | SALARY UPDATED |
| 3        | 131    | 40000.00   | 45000.00   | SALARY UPDATED |

## TEST EDGE CASE – SALARY UNCHANGED

```
UPDATE Employee
SET salary = 45000
WHERE emp_id = 131;
```

No new audit record is created

## TEST EDGE CASE – NULL SALARY

```
DROP TRIGGER validate_employee_salary;
```

```
DELIMITER //
```

```
CREATE TRIGGER validate_employee_salary
BEFORE INSERT ON Employee
FOR EACH ROW
BEGIN
```

```

        IF NEW.salary IS NULL OR NEW.salary < 15000 THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT =
                'Salary cannot be NULL and must be at least 15000';
        END IF;

END //

DELIMITER ;

INSERT INTO Employee
(emp_id, emp_name, gender, salary, hire_date, dept_id)
VALUES
(132, 'Null Salary', 'F', NULL, '2026-09-25', 1);

```

## OUTPUT:



## CHECK ALL TRIGGERS

```
SHOW TRIGGERS;
```

## OUTPUT:

The screenshot shows the MySQL IDE interface. The left sidebar displays the 'EXPLORER' panel with a search bar and a list of tables: 'demo', 'department', 'department\_salary\_summary', 'employee', 'employee\_basic', 'employee\_hierarchy', and 'employee\_project'. The main editor area shows 'Query 1' with the following SQL code:

```

1 SHOW TRIGGERS;

```

Below the query editor, the output of the command is displayed in a table format:

| Trigger                         | Event  | Table    | Statement                                                                                                                                       |
|---------------------------------|--------|----------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| validate_employee_salary        | INSERT | employee | BEGIN IF NEW.salary IS NULL OR NEW.salary < 15000 THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Salary cannot be NULL and must be at         |
| validate_employee_salary_update | UPDATE | employee | BEGIN IF NEW.salary < 15000 THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Salary must be at least 15000'; END IF; END                        |
| salary_audit_trigger            | UPDATE | employee | BEGIN IF OLD.salary <> NEW.salary THEN INSERT INTO Salary_Audit ( emp_id, old_salary, new_salary, action ) VALUES ( NEW.emp_id, OLD.salary, NEW |

CHECK STORED PROCEDURE

```
SHOW PROCEDURE STATUS
WHERE Db = 'db_454577wwy';
```

OUTPUT:

MySQL - db\_454577wwy

Query 1Query 2Query 3

1 SHOW PROCEDURE STATUS  
2 WHERE Db = 'db\_454577wwy';  
3

EXPLORER

Search...

TABLES

demo

department

department\_salary\_summary

employee

| Db           | Name              | Type      | Language | Definer          | Modified                 | Created                  | Security_type | Comment | character_set_client | collati |
|--------------|-------------------|-----------|----------|------------------|--------------------------|--------------------------|---------------|---------|----------------------|---------|
| db_454577wwy | transfer_employee | PROCEDURE | SQL      | user_454577wwy@% | 2026-09-24T21:39:54.000Z | 2026-09-24T21:39:54.000Z | DEFINER       |         | utf8mb4              | utf8mi  |